

Dissertation Proposal: Autonomous System Vulnerability Remediation

Project Summary

Overview

The rapid expansion of interconnected devices and the cloud-native paradigm has intensified the complexity and scale of cyber threats, demanding advanced security frameworks that can not only detect but autonomously adapt to and remediate evolving vulnerabilities. While Deep Learning-powered Network Intrusion Detection Systems (NIDS) and automated vulnerability scanners have emerged as powerful tools for identifying anomalous activity and system flaws, the remediation of these issues remains a predominantly manual, high-latency, and error-prone process. Existing automated solutions, primarily derived from static Automated Program Repair (APR) techniques, suffer from a critical lack of contextual awareness regarding system state, an inability to verify the safety of generated patches in runtime environments, and poor generalization to heterogeneous operating systems and cloud architectures. These limitations hinder the trust, interpretability, and real-world deployment of AI-driven remediation solutions, leaving critical infrastructure exposed to "one-day" exploits for dangerously long periods.

To address these critical shortcomings, we propose developing **Self-Healing, Verifiable, and Adaptive Autonomous Agents** for system vulnerability remediation. Our research will bridge the gap between theoretical AI advancements—specifically Large Language Models (LLMs) and Neurosymbolic AI—and real-world security needs by designing novel agentic architectures that integrate **hierarchical multi-agent reinforcement learning (H-MARL)**, **formal logic planning (PDDL)**, and **lifelong learning mechanisms**. Unlike traditional software repair, which targets source code in isolation, our focus is on **System Remediation**: the autonomous reconfiguration of operating systems, network fabrics, and cloud infrastructure to mitigate vulnerabilities without disrupting service availability.

Objectives

Our research is structured around three primary objectives, designed to culminate in submissions to NDSS 2026 and USENIX Security 2027:

1. **To develop a Neurosymbolic Planning Framework for Verifiable Remediation:** We will integrate LLM-based generative reasoning with formal verification tools (SMT solvers and PDDL planners) to mathematically guarantee that generated remediation plans satisfy safety invariants before execution. This addresses the stochastic "hallucination" risks of current LLMs.¹
2. **To engineer Hierarchical Multi-Agent Architectures for Cross-Domain Resilience:** We will design a system of specialized agents—scanners, planners, and executors—coordinated by a meta-controller to handle complex, multi-step remediation tasks that exceed the context window and reasoning capabilities of single-agent

models.³

3. **To implement Lifelong Learning with Hybrid Memory Systems:** We will create agents capable of continuous adaptation without catastrophic forgetting, utilizing a hybrid memory architecture (Vector Database + Knowledge Graph) to persist remediation strategies across diverse environments and attack vectors.⁵

Intellectual Merit

This research bridges cybersecurity, software engineering, and artificial intelligence to tackle the asymmetry between automated attack and manual defense. By translating concepts from Automated Program Repair (APR)—such as fault localization and patch generation—to the domain of System Administration, we address a fundamental gap in the literature: the lack of "state-aware" repair mechanisms. Our novel approach incorporates Neurosymbolic AI to impose logical bounds on neural generation, ensuring that autonomous agents operate within strict safety guardrails. Furthermore, we introduce the concept of "Adversarial Hardening for Agents," evaluating how remediation agents can be defended against "counter-remediation" attacks where adversaries poison system logs to hijack the repair process.⁷ This work contributes to the theoretical foundations of Trustworthy Agentic AI, establishing new standards for verifiable autonomy in high-stakes environments.

Broader Impacts of the Proposed Work

This research advances the science of Autonomous Cyber Defense (ACD) by creating systems that can self-heal at machine speed, drastically reducing the window of exposure for critical infrastructure. Ensuring model transparency through explainable planning (XAI) will increase trust and adoption of AI-powered security solutions in sectors such as healthcare, finance, and defense. We will disseminate our findings through top-tier conferences (NDSS, USENIX Security), academic journals, and open-source repositories, including a novel benchmark suite, SysRepair-Bench, for evaluating agentic remediation. To engage the community and align with Cisco's mission of inclusive education, we will develop curriculum modules on "AI for Security Operations" and partner with the Mayborn Museum at Baylor University to create interactive exhibits demonstrating the concept of "Digital Immunology," inspiring the next generation of diverse talent in STEM and cybersecurity fields.⁹

Project Description

1. Research Summary

Objective 1: To develop a Neurosymbolic Planning Framework for Verifiable Remediation (Target: NDSS 2026)

We will focus on overcoming the "probabilistic correctness" problem of LLMs. Current Large Language Models can generate plausible-looking remediation commands (e.g., Ansible playbooks or Bash scripts), but they lack an internal model of correctness or safety. We propose a Neurosymbolic Architecture where an LLM functions as a "translator" that converts natural language vulnerability descriptions and system states into Planning Domain Definition

Language (PDDL). A symbolic planner then generates a logically sound sequence of actions, which is verified against safety constraints (e.g., "Service X must remain reachable") before being translated back into executable code. This hybrid approach ensures that the agent's actions are verifiable by design, fostering the trust required for autonomous operations in production environments.¹

Objective 2: To engineer Hierarchical Multi-Agent Architectures for Cross-Domain Resilience (Target: USENIX Security 2027)

Complex system vulnerabilities often span multiple layers—from application code to OS configurations and network policies. A single agent with a limited context window cannot effectively diagnose and remediate such interconnected issues. We will design a Hierarchical Multi-Agent System (H-MAS) where a high-level "Orchestrator Agent" decomposes remediation goals into sub-tasks for specialized "Worker Agents" (e.g., a Linux Hardening Agent, a Cloud Policy Agent, a Network Forensics Agent). We will investigate optimal inter-agent communication protocols and shared memory structures (specifically Knowledge Graphs) to enable collaboration without context overflow. This objective explicitly targets the scalability and robustness required for enterprise-grade autonomous defense.¹¹

Objective 3: To implement Lifelong Learning with Hybrid Memory Systems

Aligned with Cisco's Request for Proposals on LLM-Based Autonomous Agents, we will address the challenge of Catastrophic Forgetting in lifelong learning scenarios. As remediation agents encounter new vulnerabilities (zero-days) and environments, they must integrate new knowledge without overwriting previously learned strategies. We will develop a Hybrid Memory System that combines Vector Databases (for semantic retrieval of unstructured threat intel) and Knowledge Graphs (for structured representation of system dependencies and past successful fixes). This memory architecture will support Experience Replay and In-Context Learning, allowing agents to adapt to heterogeneous domains (e.g., moving from Ubuntu to Alpine Linux, or AWS to Azure) with minimal retraining.⁵

2. Motivation and Significance

The cybersecurity landscape is currently defined by a "Speed of Action" gap. Attackers leverage automation to exploit vulnerabilities within minutes of disclosure, while defenders typically require days or weeks to patch systems. The **2025 State of Vulnerability Remediation** report highlights that fewer than 10% of organizations are confident in their ability to remediate vulnerabilities in a timely manner, with manual processes and "tool sprawl" cited as primary bottlenecks.¹⁴ The result is a persistent "**Security Debt**," where known vulnerabilities remain unpatched due to the sheer volume of alerts and the operational risks associated with applying fixes manually.¹⁵

The Limitations of Traditional Automation

Traditional automated remediation tools (e.g., patch management systems) are rigid and rule-based. They function well for homogenous updates (e.g., updating Chrome across a fleet) but fail catastrophically when faced with context-dependent vulnerabilities. For instance, a vulnerability in a web server might require a specific configuration change rather than a simple binary update, or a patch might need to be applied in a specific order across a

cluster to avoid downtime. Traditional scripts cannot "reason" about these dependencies; they simply execute or fail.

The Promise and Peril of LLMs

Large Language Models (LLMs) offer a potential solution due to their ability to understand context, synthesize information from disparate sources (documentation, logs, CVE reports), and generate code. Recent studies have shown LLMs' potential in Automated Program Repair (APR) for software code.¹⁶ However, applying LLMs directly to System Remediation introduces severe risks:

1. **Hallucination:** LLMs may invent non-existent package names, command flags, or configuration parameters, leading to failed remediation or, worse, system corruption.¹⁸
2. **Lack of State Awareness:** An LLM does not inherently "know" the current state of a system (e.g., whether a service is running, or a file exists) unless that state is explicitly provided in the prompt. Managing this state tracking over long remediation sessions is a major challenge.¹
3. **Safety & Reversibility:** Unlike a software patch that can be reverted in Git, a system command (e.g., `rm -rf`, `iptables -F`) can have immediate, irreversible consequences. LLMs lack the foresight to inherently understand these side effects.

Significance of Neurosymbolic and Multi-Agent Approaches

To harness the power of LLMs while mitigating their risks, a paradigm shift is required.

Neurosymbolic AI offers a mechanism to bound the "creativity" of LLMs with the "correctness" of formal logic. By using LLMs to generate plans within a formal language like PDDL, and using symbolic solvers to validate those plans, we can achieve provable safety properties.¹⁹

Furthermore, Multi-Agent Systems allow for the decomposition of complex tasks. Just as a human SOC has Tier 1, 2, and 3 analysts, an automated defense system should have specialized agents for triage, planning, and execution. This division of labor not only improves scalability but also allows for "Human-on-the-loop" oversight, where critical decisions proposed by agents are verified by humans before execution.³

This research is significant because it moves beyond the *detection* of threats—a problem largely addressed by current commercial tools—to the *remediation* of threats, the "last mile" of cybersecurity that remains the most labor-intensive and risky. By automating this phase with verifiable, adaptive agents, we can fundamentally alter the economics of cyber defense, making it scalable and responsive enough to counter AI-driven attacks.

3. Background

Automated Program Repair (APR) Concepts

Research into software vulnerability remediation provides a foundational toolkit for our work. APR techniques generally fall into three categories: heuristic/search-based (e.g., GenProg), constraint-based (e.g., SemFix), and learning-based (using LLMs).

- *Transferable Concept: Fault Localization.* In software, this involves identifying the specific lines of code causing a bug. In system administration, we must adapt this to **Configuration Localization**—identifying the specific lines in a config file (e.g.,

nginx.conf, /etc/ssh/sshd_config) or the specific cloud resource attributes (e.g., AWS Security Group rules) that constitute the vulnerability.¹⁷

- *Transferable Concept: **Generate-and-Validate***. APR generates candidate patches and validates them against a test suite. For systems, we must define "**System Test Oracles**"—scripts that verify a vulnerability is closed (e.g., using a PoC exploit) *and* that system functionality is preserved (regression testing).²²

Neurosymbolic AI in Cybersecurity

Neurosymbolic AI integrates neural networks with symbolic reasoning. This is particularly relevant for cybersecurity, where we have both unstructured data (logs, threat intel reports) and structured constraints (firewall rules, access policies).

- *Current State:* Approaches like **PDDL-Instruct**¹ and **LLM+P**²³ have demonstrated that LLMs can translate natural language problems into PDDL, which a classical planner then solves. This ensures the generated plan is logically sound. In cybersecurity, this has been applied to **pentesting planning**²⁴, but its application to *defensive remediation* and *configuration repair* is underexplored. We aim to close this gap.

Multi-Agent Reinforcement Learning (MARL) for Defense

Autonomous Cyber Defense (ACD) is often modeled as a game between attackers and defenders. CybORG (Cyber Operations Research Gym) 25 is a standard environment for training RL agents in network defense.

- *Challenge:* Most MARL research focuses on abstract, high-level actions (e.g., "Block Subnet") rather than the granular, realistic command-line remediation required in practice. Furthermore, existing agents often suffer from non-stationarity and lack of coordination.⁴ Our proposal introduces **Hierarchical MARL** to handle the varying levels of abstraction, from high-level strategy to low-level execution.¹¹

Lifelong Learning and Catastrophic Forgetting

In dynamic system environments, agents must continuously learn. Catastrophic forgetting occurs when fine-tuning an agent on new tasks erases its knowledge of old ones.²⁷

- *Proposed Solution:* Instead of relying solely on updating model weights, we propose **Retrieval-Augmented Generation (RAG)** using a **Knowledge Graph** as a form of "external memory." This aligns with recent findings that hybrid memory systems (vector + graph) significantly enhance agent consistency and long-term reasoning.¹³

4. Research Questions & Objectives

Objective 1: To Study Neurosymbolic Planning Frameworks for Verifiable System Remediation

Targeting NDSS 2026

Challenges & Questions:

- *How can we bridge the semantic gap between unstructured vulnerability reports (CVEs) and formal state transitions in system administration?*
- *Can we formalize system administration tasks (e.g., "patching openssl") as PDDL domains to guarantee the logical correctness of remediation plans?*
- *How does a neurosymbolic verifier compare to standard LLM "self-correction" in reducing the rate of hazardous commands (e.g., breaking dependencies)?*

Motivation:

Current LLM agents operating in "chat" modes often produce syntactically correct but semantically disastrous commands. They lack a rigorous internal model of cause and effect in a stateful system. By enforcing a neurosymbolic bottleneck—where the LLM generates a symbolic plan that must be validated by a logic solver before execution—we can mathematically guarantee that the proposed remediation path does not violate defined safety invariants (e.g., "Invariant: CriticalService must be active"). This objective directly addresses the Safety and Robustness requirements of the Cisco funding criteria.⁵

Research Plan:

- **Task 1a: Formalizing the "SysAdmin" Domain.** We will develop a **PDDL Domain Definition** for Linux and Cloud system administration. This involves defining Types (e.g., Service, File, Package, Port), Predicates (e.g., (is_installed?p), (is_running?s), (port_open?p)), and Actions (e.g., install_package, modify_config, restart_service) with precise preconditions and effects.
- **Task 1b: LLM-based Translation & Planning.** We will fine-tune a specialized LLM (e.g., Llama-3 or Mistral) to translate unstructured CVE remediation advice and system audit logs into **PDDL Problem Files** (Initial State and Goal State).
- **Task 1c: The Neurosymbolic Loop.** We will implement the **Logic-Generate-Verify** loop:
 1. **Translation:** Agent perceives system state and CVE, translates to PDDL.
 2. **Planning:** A classical planner (e.g., Fast Downward) attempts to find a valid plan. If no plan exists (e.g., conflicting constraints), the symbolic solver returns the specific error to the LLM for re-translation/refinement.
 3. **Code Synthesis:** Once a valid symbolic plan is found (e.g., step1: stop_service, step2: update_config), the LLM translates these symbolic steps into concrete executable code (e.g., Ansible tasks or Bash commands).
 4. **Verification:** A **Constraint Solver (SMT)** checks the generated code against safety policies (e.g., "No command shall execute rm -rf on /").²⁹

Evaluation:

We will evaluate this framework using the SysRepair-Bench (detailed in Section 5). The primary metric will be the Verification Pass Rate (how often the generated plan is logically valid) and the Safety Violation Rate (how often the agent attempts a dangerous action). We hypothesize that the neurosymbolic approach will reduce safety violations by >90% compared to a standard ReAct agent.³¹

Objective 2: Explore Hierarchical Multi-Agent Frameworks for Cross-Domain

Resilience

Targeting USENIX Security 2027

Challenges & Questions:

- *How can we effectively decompose complex, multi-stage vulnerability remediation tasks into sub-problems for specialized agents?*
- *Does a hierarchical architecture (Manager-Worker) outperform a flat multi-agent debate structure in resolving inter-component dependencies?*
- *Can a shared "Memory-Knowledge Graph" mitigate the context window limitations when agents collaborate over long horizons?*

Motivation:

Real-world enterprise environments are heterogeneous. A vulnerability might exist in a web application (Java), requiring a fix that impacts the database (SQL) and the container orchestrator (Kubernetes). A single generalist agent struggles with the specific nuances of each domain. Hierarchical Multi-Agent Systems (H-MAS) allow for specialization and coordinated action. By mimicking a human security team structure, we can achieve greater scalability and resilience.³

Research Plan:

- **Task 2a: Agent Specialization & Taxonomy.** We will define a taxonomy of agents based on the **NIST Cybersecurity Framework**:
 - **Scanner Agent (Identify):** Specialized in using tools like Nmap, OpenVAS, and reading logs. It populates the "World State."
 - **Analyst Agent (Protect/Detect):** analyzing the root cause and proposing a remediation strategy (using the PDDL planner from Obj 1).
 - **Engineer Agent (Respond/Recover):** Specialized in generating platform-specific code (e.g., one agent for Linux, one for AWS, one for K8s).
 - **QA Agent (Verify):** Responsible for executing test cases and verifying the fix.³²
- **Task 2b: Graph-Based Shared Memory.** Agents will collaborate not by passing massive text logs, but by reading and writing to a shared **Dynamic Knowledge Graph**. This graph represents the "Mental Model" of the system (e.g., Node: WebServer -> HasVulnerability: CVE-2024-XYZ -> DependsOn: LibSSL). This addresses the **Catastrophic Forgetting** and context limit problems, allowing agents to maintain situational awareness over long operational periods.¹³
- **Task 2c: Hierarchical Orchestration.** We will implement a **Meta-Controller Agent** using **Hierarchical Reinforcement Learning (HRL)**. This manager learns a policy for *task delegation*—deciding which specialized agent is best suited for a sub-problem based on the current state of the Knowledge Graph.⁴

Evaluation:

We will deploy this H-MAS in the Cyborg environment (CAGE Challenge scenarios) extended with realistic remediation tasks. We will measure Collaboration Overhead (communication

cost), Remediation Success Rate across multi-vector attacks, and Resilience to agent failure (e.g., if the Linux agent fails, can the Cloud agent compensate by isolating the instance?).²⁵

Objective 3: Investigate Safety-by-Design and Adversarial Robustness

Transferable Concepts & Lifelong Learning

Challenges & Questions:

- *Are autonomous remediation agents vulnerable to "Indirect Prompt Injection" via poisoned logs or malicious configuration files?*
- *How can we implement "Cognitive Guardrails" that prevent agents from being socially engineered by an attacker present in the network?*

Motivation:

Autonomous agents are a double-edged sword. If an attacker knows an agent is monitoring logs, they can inject malicious strings (e.g., User: admin; DROP TABLE users;) into the logs, hoping the agent's LLM processes it as a command. This "Counter-Remediation" attack vector is a critical blind spot in current research. Ensuring the agent is robust to such adversarial inputs is a prerequisite for deployment.⁸

Research Plan:

- **Task 3a: Red Teaming Agents.** We will systematically generate adversarial datasets (poisoned logs, malicious CVE descriptions) to test the agent's robustness. This aligns with the **"Agent Security Bench" (ASB)** framework.⁷
- **Task 3b: Developing Cognitive Guardrails.** We will design a **"Monitor-Shield" architecture**. A separate, smaller, and highly constrained model (or symbolic policy) acts as a "Shield," filtering the inputs and outputs of the main agent. It enforces non-negotiable safety rules (e.g., "Remediation commands cannot modify /etc/shadow directly").³⁶
- **Task 3c: Lifelong Learning via Graph Replay.** To ensure continuous improvement and prevent regression, we will implement **Graph-Based Experience Replay**. Successful remediation traces (State-Action-Outcome triples) are stored in the Knowledge Graph. When facing a new but similar vulnerability, the agent retrieves these past successful "sub-plans" to guide its reasoning, effectively realizing **Few-Shot Learning** from its own history.⁶

5. Evaluation

The success of this research will be rigorously evaluated through a multi-faceted approach, combining novel benchmark creation with standardized simulation environments.

1. SysRepair-Bench (Novel Contribution)

Existing benchmarks like SWE-bench focus on software repositories³⁹, and OSWorld focuses on GUI interactions.⁴⁰ Neither adequately captures the system administration and remediation domain. We will construct SysRepair-Bench, a dataset of 500+ reproducible

system vulnerability scenarios.

- **Composition:**
 - **OS Level:** Misconfigured SSH, Weak file permissions, Kernel parameter tuning (sysctl), Vulnerable package versions (requiring complex upgrades).
 - **Network Level:** Open firewall ports, Misconfigured DNS, Rogue services.
 - **Cloud Level:** S3 bucket exposure, IAM privilege escalation paths, Kubernetes RBAC misconfigurations.
- **Source:** We will mine data from **LinuxFLBench**⁴¹, **VulnRepairEval**²², and real-world CVEs with available PoCs.
- **Metric:** We will use "**Proof of Remediation**" (**PoR**). For each scenario, we define a verification script (e.g., an InSpec profile or a PoC exploit). A remediation is "Success" only if the PoC fails AND the service passes a regression test (functionality check).²²

2. CybORG Simulation (Adversarial & Multi-Agent)

We will utilize CybORG (Cyber Operations Research Gym), specifically building upon the CAGE Challenge scenarios.²⁵

- **Scenario:** A simulated enterprise network with Red Agents (Attackers) and Blue Agents (our Remediation H-MAS).
- **Metrics:**
 - **Mean Time to Remediate (MTTR):** Time from detection to verified fix.
 - **Service Availability:** Percentage of uptime for "Green Agents" (users) during the remediation process.
 - **Agent Stability:** Frequency of "flapping" (repeatedly applying and reverting fixes).
 - **Safety Score:** Number of "unsafe" actions attempted (blocked by the Shield).⁴

3. Comparative Analysis

We will compare our Neurosymbolic H-MAS against:

- **Baselines:** Standard ReAct Agents (GPT-4o), AutoGPT, and specialized single-agent tools like **AutoPentest** (adapted for defense).¹⁸
- **Ablation Studies:** We will test the system *without* the PDDL planner and *without* the Knowledge Graph to quantify the contribution of each component to the overall success rate and safety.³¹

6. Project Timeline and Personnel

Year 1: Foundations & Neurosymbolic Planning (Target: NDSS 2026)

- **Q1-Q2:** Comprehensive Literature Review. Development of **SysRepair-Bench** (mining CVEs, creating Dockerized environments). Data collection for fine-tuning Planner LLMs.
- **Q3:** Implementation of **Objective 1** (Neurosymbolic Planner). Development of the "System-to-PDDL" translator and the Symbolic Verification Loop.
- **Q4:** Evaluation on SysRepair-Bench (Single-Host Scenarios). Drafting and submission of paper to **NDSS 2026**: "*Logic-Bounded Agents: Verifiable Planning for Autonomous*

System Hardening."

Year 2: Scaling to Multi-Agent Architectures (Target: USENIX Security 2027)

- **Q1:** Implementation of **Objective 2** (H-MAS). Development of the Agent Taxonomy and the Shared Knowledge Graph (GraphRAG) architecture.
- **Q2:** Integration with **CybORG**. Training of the Hierarchical Orchestrator (Meta-Controller) using MARL.
- **Q3:** Large-scale evaluation on complex, multi-host scenarios. Adversarial testing (Red Teaming) for **Objective 3**.
- **Q4:** Drafting and submission of paper to **USENIX Security 2027: "Orchestrating Resilience: Hierarchical Multi-Agent Systems for Verifiable Cloud Self-Healing."**

Year 3: Lifelong Learning, Safety & Tech Transfer

- **Q1-Q2:** Refinement of **Lifelong Learning** mechanisms (Graph Experience Replay). Deep analysis of "Counter-Remediation" attacks and Guardrail effectiveness.
- **Q3:** Integration of all components into a unified, open-source framework. Development of educational modules for the **Mayborn Museum** collaboration.
- **Q4:** Final Dissertation writing and defense.

Personnel:

- **Principal Investigator (PhD Candidate):** Responsible for architecture design, model training, and primary experimentation.
- **Faculty Advisor:** Providing oversight on formal verification methods and research strategy.
- **Collaborators:** Engaging with Cisco researchers for industry alignment and potentially engaging undergraduate students for benchmark dataset curation.

7. Intellectual Merit

This research addresses the fundamental fragility of current AI systems in high-stakes domains.

1. **Advancing Trustworthy AI:** By coupling the generative power of LLMs with the rigorous bounds of **Formal Methods (PDDL/SMT)**, we propose a solution to the "Hallucination Problem" in system administration. This **Neurosymbolic** approach provides a blueprint for building AI agents that are not just "smart" but *safe* and *verifiable*.¹⁹
2. **Theoretical Contributions to Agentic AI:** We formalize the problem of **System Remediation** as a distinct class of planning problem, separating it from software repair. Our work on **Hierarchical Multi-Agent** coordination with **Graph-based Memory** contributes to the broader field of Distributed AI, offering novel solutions to the context-window bottleneck and catastrophic forgetting.¹¹
3. **Transfer Learning:** By adapting **Automated Program Repair (APR)** techniques (fault localization, patch validation) to the *system* domain, we create a unified theory of "Digital

Repair" that spans code and infrastructure.²¹

8. Broader Impacts

National Security and Infrastructure Resilience:

The capability to autonomously remediate vulnerabilities at machine speed is a critical strategic asset. This research directly supports the security of national critical infrastructure (energy grids, financial networks, cloud backbones) against automated, AI-driven cyber-attacks. By reducing the "Mean Time to Remediate" from weeks to minutes, we drastically shrink the attack surface available to adversaries.¹²

Democratization of Cyber Defense:

Small and Medium Enterprises (SMEs) often lack the resources to staff a 24/7 Security Operations Center (SOC). The open-source Autonomous Remediation Agents developed in this project could provide these organizations with an "AI SOC," leveling the playing field against sophisticated threat actors.

Education and Public Outreach:

We are committed to demystifying AI and cybersecurity for the public.

- **Curriculum Development:** We will integrate findings into university courses, training the next generation of "AI-Native" security practitioners who understand how to collaborate with and govern autonomous agents.
- **Mayborn Museum Partnership:** We will collaborate with the **Mayborn Museum at Baylor University** to design an interactive exhibit titled *"The Digital Immune System."* This exhibit will visualize how autonomous agents detect and "heal" a network in real-time, using accessible metaphors to inspire K-12 students (specifically targeting underrepresented groups in STEM) to pursue careers in cybersecurity and AI.⁹

Open Science:

We will release SysRepair-Bench, the Neurosymbolic Planner code, and our Agent Safety Guardrails as open-source resources. This promotes reproducibility and accelerates innovation across the global research community.

Detailed Research Methodology

Objective 1: Neurosymbolic Planning (L-Gen-V Loop)

The core innovation is the **Logic-Generate-Verify (L-Gen-V)** loop:

1. **State Abstraction:** The agent uses tools (osquery, kubectl) to perceive the system. An LLM acts as a "Semantic Transpiler," converting raw JSON outputs into a formal PDDL State (e.g., (is-vulnerable apache-2.4), (port-open 80)).
2. **Plan Synthesis:** A "Planner LLM" generates a high-level PDDL plan (e.g., step1: isolate_host, step2: patch_service, step3: verify_config, step4: reconnect).
3. **Symbolic Verification:** A classical planner (e.g., **Fast Downward**) verifies if this plan is *feasible* given the domain constraints (e.g., "Cannot patch without root privileges"). If

invalid, it returns a symbolic error trace.

4. **Iterative Refinement:** The LLM uses the error trace to self-correct the plan. This **Iterative Neurosymbolic Refinement (INR)** continues until a valid plan is found or a timeout occurs.²³
5. **Concrete Execution:** Only the *verified* plan is translated into executable scripts (Ansible/Bash) for the specific target OS.

Objective 2: H-MAS with Graph Memory

We employ a **Cognitive Architecture** for the multi-agent system:

- **Perception:** Agents write observations to a shared **Knowledge Graph (Neo4j)** rather than a chat log. This structures the data (e.g., Node:Server1 --> CVE-2025-123).
- **Memory:** We implement **GraphRAG**. When an agent needs to solve a problem, it queries the graph not just for keywords (Vector Search) but for *relationships* (e.g., "Find all past remediation plans for vulnerabilities affecting libssl on Ubuntu"). This provides deep, structural context that prevents agents from repeating past mistakes.¹³
- **Coordination:** The **Orchestrator Agent** uses a **Hierarchical RL** policy to assign tasks. It learns a "Manager Policy" (e.g., "If high severity, assign to Senior Patch Agent; if low severity, assign to Junior Agent").¹¹

Objective 3: Adversarial Robustness

We define a threat model where the agent itself is the target.

- **Attack: Indirect Prompt Injection.** An attacker places a file named ; reboot; or creates a log entry with a hidden instruction: ``.
- **Defense:** We implement **Input Sanitization Layers** and **Output Filters**. The "Guardrail Agent" uses a non-LLM based parser (e.g., AST analysis of the generated Bash script) to detect and block dangerous system calls before they leave the agent's sandbox. We essentially build a "Firewall for Cognition".³⁵

References:

1

Works cited

1. ML Tea: PDDL-Instruct: Enhancing Symbolic Planning Capabilities in LLMs through Logical Chain-of-Thought Instruction Tuning / Incentive-Aware Dynamic Pricing for Constrained Resource Allocation with Strategic Agents | MIT CSAIL, accessed December 23, 2025, <https://www.csail.mit.edu/event/ml-tea-pddl-instruct-enhancing-symbolic-planning-capabilities-llms-through-logical-chain>
2. Towards Formal Verification of LLM-Generated Code from Natural Language

- Prompts, accessed December 23, 2025, <https://arxiv.org/html/2507.13290v2>
3. Composable Multi-Tool Pipelines for AI Agents in DevSecOps: Architecture, Orchestration, and Evaluation - ResearchGate, accessed December 23, 2025, https://www.researchgate.net/publication/397171871_Composable_Multi-Tool_Pipelines_for_AI_Agents_in_DevSecOps_Architecture_Orchestration_and_Evaluation
 4. Hierarchical Multi-agent Reinforcement Learning for Cyber Network Defense, accessed December 23, 2025, https://rlj.cs.umass.edu/2025/papers/RLJ_RLC_2025_77.pdf
 5. LLM-Based Autonomous Agents and Agentic Systems - Cisco Research Home, accessed December 23, 2025, <https://research.cisco.com/open-rfps/23>
 6. Memory-Augmented Large Language Model-Based Agent with Cross-Task Experience Learning | OpenReview, accessed December 23, 2025, <https://openreview.net/forum?id=OGMt2OWeCb>
 7. Agent Security Bench (ASB) - Emergent Mind, accessed December 23, 2025, <https://www.emergentmind.com/topics/agent-security-bench-asb>
 8. OWASP Top 10 LLM, Updated 2025: Examples & Mitigation Strategies - Oligo Security, accessed December 23, 2025, <https://www.oligo.security/academy/owasp-top-10-llm-updated-2025-examples-and-mitigation-strategies>
 9. Proposal__Rivas__El_Mahmoud.pdf
 10. From an LLM Swarm to a PDDL-empowered Hive: Planning Self-executed Instructions in a Multi-modal Jungle - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2412.12839v2>
 11. Hierarchical Reinforcement Learning, Sequential Behavior, and the Dorsal Frontostriatal System - PMC - NIH, accessed December 23, 2025, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9274316/>
 12. Autonomous cyber defence: Beyond games? - The Alan Turing Institute, accessed December 23, 2025, <https://www.turing.ac.uk/sites/default/files/2024-07/hicks-mavroudis-2024.pdf>
 13. Vector vs. Graph RAG: How to Actually Architect Your AI Memory - Optimum Partners, accessed December 23, 2025, <https://optimumpartners.com/insight/vector-vs-graph-rag-how-to-actually-architect-your-ai-memory/>
 14. 2025 State of Vulnerability Remediation Report: Manual Processes and Low Confidence, accessed December 23, 2025, <https://mondoo.com/blog/2025-state-of-vulnerability-remediation-reveals-manual-processes-and-low-confidence>
 15. State of Software Security (SoSS) 2025: A New View of Maturity - Veracode, accessed December 23, 2025, <https://www.veracode.com/resources/analyst-reports/state-of-software-security-2025/>
 16. Automated Program Repair in the Era of Large Pre-trained Language Models - Lingming Zhang, accessed December 23, 2025, <https://lingming.cs.illinois.edu/publications/icse2023a.pdf>
 17. Automated Program Repair, accessed December 23, 2025,

- <https://par.nsf.gov/servlets/purl/10135834>
18. AutoPentest: Enhancing Vulnerability Management With Autonomous LLM Agents, accessed December 23, 2025, <https://www.alphaxiv.org/overview/2505.10321v1>
 19. An Introduction of Neuro-Symbolic AI and Its Significance for Cybersecurity, accessed December 23, 2025, <https://www.rsaconference.com/library/blog/an-introduction-of-neuro-symbolic-ai-and-its-significance-for-cybersecurity>
 20. ATA: A Neuro-Symbolic Approach to Implement Autonomous and Trustworthy Agents - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2510.16381v1>
 21. Advancements in automated program repair: a comprehensive review - ResearchGate, accessed December 23, 2025, https://www.researchgate.net/publication/389601152_Advancements_in_automated_program_repair_a_comprehensive_review
 22. VulnRepairEval: An Exploit-Based Evaluation Framework for Assessing Large Language Model Vulnerability Repair Capabilities - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2509.03331v1>
 23. Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning | OpenReview, accessed December 23, 2025, <https://openreview.net/forum?id=zDbsSscmuj>
 24. Automated Penetration Testing with LLM Agents and Classical Planning - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2512.11143v1>
 25. [論文評述] CybORG++: An Enhanced Gym for the Development of Autonomous Cyber Agents - Moonlight, accessed December 23, 2025, <https://www.themoonlight.io/tw/review/cyborg-an-enhanced-gym-for-the-development-of-autonomous-cyber-agents>
 26. [2108.09118] CybORG: A Gym for the Development of Autonomous Cyber Agents - arXiv, accessed December 23, 2025, <https://arxiv.org/abs/2108.09118>
 27. Continual Learning and Catastrophic Forgetting: The Challenges and Strategies in AI | by Siddhartha Pramanik | Medium, accessed December 23, 2025, <https://medium.com/@siddharthapramanik771/continual-learning-and-catastrophic-forgetting-the-challenges-and-strategies-in-ai-636e79a6a449>
 28. Graphiti: Knowledge Graph Memory for an Agentic World - Neo4j, accessed December 23, 2025, <https://neo4j.com/blog/developer/graphiti-knowledge-graph-memory/>
 29. terraform validate command reference - HashiCorp Developer, accessed December 23, 2025, <https://developer.hashicorp.com/terraform/cli/commands/validate>
 30. Validate your infrastructure in Terraform's configuration language - HashiCorp Developer, accessed December 23, 2025, <https://developer.hashicorp.com/terraform/language/validate>
 31. Agentic Program Repair from Test Failures at Scale: A Neuro-symbolic approach with static analysis and test execution feedback - arXiv, accessed December 23, 2025, <https://arxiv.org/pdf/2507.18755>
 32. AI-driven Security as Code for software development using multi-agent systems

- CEUR-WS.org, accessed December 23, 2025, <https://ceur-ws.org/Vol-4024/paper13.pdf>
- 33. Bring Your Enterprise Information Security Management System with GraphRAG to the Next Level - Graphwise, accessed December 23, 2025, <https://graphwise.ai/blog/bring-your-enterprise-information-security-management-system-with-graphrag-to-the-next-level/>
- 34. Large Language Models are Autonomous Cyber Defenders - arXiv, accessed December 23, 2025, <https://arxiv.org/pdf/2505.04843>
- 35. Agentic Autonomy Levels and Security | NVIDIA Technical Blog, accessed December 23, 2025, <https://developer.nvidia.com/blog/agentic-autonomy-levels-and-security/>
- 36. Neurosymbolic Reinforcement Learning with Formally Verified Exploration, accessed December 23, 2025, <https://proceedings.neurips.cc/paper/2020/file/448d5eda79895153938a8431919f4c9f-Paper.pdf>
- 37. Safe Reinforcement Learning for Autonomous Lane Changing Using Set-Based Prediction - mediaTUM, accessed December 23, 2025, <https://mediatum.ub.tum.de/doc/1548735/256213.pdf>
- 38. Continual Learning: Overcoming Catastrophic Forgetting in Neural Networks, accessed December 23, 2025, https://www.researchgate.net/publication/390172499_Continual_Learning_Overcoming_Catastrophic_Forgetting_in_Neural_Networks
- 39. What skills does SWE-bench Verified evaluate? | Epoch AI, accessed December 23, 2025, <https://epoch.ai/blog/what-skills-does-swe-bench-verified-evaluate>
- 40. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments, accessed December 23, 2025, <https://os-world.github.io/>
- 41. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2505.19489v1>
- 42. Neuro-Symbolic AI for Cybersecurity: State of the Art, Challenges, and Opportunities - arXiv, accessed December 23, 2025, <https://arxiv.org/pdf/2509.06921>
- 43. Automated Program Repair - squaresLab, accessed December 23, 2025, <https://squareslab.github.io/materials/legoues-cacm2019.pdf>
- 44. AlxCC: AI Cyber Challenge - DARPA, accessed December 23, 2025, <https://www.darpa.mil/research/programs/ai-cyber>
- 45. LOOP: A Plug-and-Play Neuro-Symbolic Framework for Enhancing Planning in Autonomous Systems - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2508.13371v1>
- 46. AI-powered patching: the future of automated vulnerability fixes - Google Research, accessed December 23, 2025, https://research.google/pubs/ai-powered-patching-the-future-of-automated-vulnerability-fixes/?Offer=ab_ss_reeng_plt_var2
- 47. Leveraging Large Language Models for Autonomous Cyber Defense: Insights from CAGE-2 Simulations - IEEE Computer Society, accessed December 23,

- 2025,
<https://www.computer.org/csdl/magazine/ex/2025/04/10991969/26y4vtjOcFy>
48. [Literature Review] Large Language Models are Autonomous Cyber Defenders - Moonlight, accessed December 23, 2025,
<https://www.themoonlight.io/en/review/large-language-models-are-autonomous-cyber-defenders>
 49. Autonomous Multi-Agent Defense - Emergent Mind, accessed December 23, 2025, <https://www.emergentmind.com/topics/autonomous-multi-agent-defense>
 50. How Multi-Agent AI Is Redefining Cloud Application Security with AI Autonomous Governance | by Valdez Ladd | Medium, accessed December 23, 2025,
https://medium.com/@oracle_43885/how-multi-agent-ai-is-redefining-cloud-application-security-with-ai-autonomous-governance-9d39c0c03b20
 51. LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations - IEEE Xplore, accessed December 23, 2025,
<https://ieeexplore.ieee.org/iel8/11025548/11025536/11025712.pdf>
 52. Neuro-Symbolic AI: The key to Safe and Reliable AI in High-Risk Sectors | by Landry Holi, accessed December 23, 2025,
<https://medium.com/@landry.holi/neuro-symbolic-ai-the-key-to-safe-and-reliable-ai-in-high-risk-sectors-ac5fc580e7f1>
 53. Introducing Cisco's Integrated AI Security and Safety Framework, accessed December 23, 2025, <https://blogs.cisco.com/ai/security-framework>
 54. Verifiable Self-Certifying Autonomous Systems - The University of Manchester, accessed December 23, 2025,
<https://personalpages.manchester.ac.uk/staff/louise.dennis/pubs/PID5539853.pdf>
 55. Building a Multi-Agent Security Pipeline with Google's A2A Protocol and GitGuardian, accessed December 23, 2025,
<https://blog.gitguardian.com/building-a-multi-agent-security-pipeline-with-google-a2a-protocol-and-gitguardian/>
 56. Shell or Nothing: Real-World Benchmarks and Memory-Activated Agents for Automated Penetration Testing - arXiv, accessed December 23, 2025,
<https://arxiv.org/html/2509.09207v2>
 57. SWE-Bench Pro (Public Dataset) - Scale AI, accessed December 23, 2025,
https://scale.com/leaderboard/swe_bench_pro_public
 58. LLMs as Hackers: Autonomous Linux Privilege Escalation Attacks - arXiv, accessed December 23, 2025, <https://arxiv.org/html/2310.11409v3>
 59. VADER: A Human-Evaluated Benchmark for Vulnerability Assessment, Detection, Explanation, and Remediation - arXiv, accessed December 23, 2025,
<https://arxiv.org/html/2505.19395v1>